



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Assignment Report: CSA1504 — Cloud Computing and Big Data Analytics

Smart Energy-Consumption Analytics Using Big Data Technologies

Student Names & Reg. No:

YADDALA KUSHAL KUMAR

(192524068)

Course Code & Name:

**CSA1504 - Cloud Computing and Big
Data Analytics**

Course Outcomes:

CO4, CO5

Bloom's Taxonomy Level:

L3 - Apply; L4 - Analyse

SDG Mapping:

**SDG 7, SDG 9, SDG 11, SDG 12, SDG
13**

Submission Date:

September 2026

1.Problem Definition

The modern energy landscape relies heavily on advanced metering infrastructure, generating a massive influx of continuous data streams. The energy utility receives a large volume of records containing information such as meter ID, building/consumer ID, date, time, energy consumption, voltage, current, power factor, appliance/category, tariff period, and renewable-energy generation. Processing such datasets using conventional centralized systems is time-consuming and makes it difficult to identify consumption patterns, peak-demand periods, inefficient usage, and opportunities for energy conservation. To resolve these severe computational bottlenecks, the proposed system shall use HDFS for distributed storage of energy datasets, MapReduce for parallel processing and analytics, and YARN for cluster resource management and job execution. By migrating from legacy relational databases to a distributed Hadoop ecosystem, energy providers can achieve the high throughput and fault tolerance necessary to design a Smart Energy-Consumption Analytics System capable of processing large-scale energy data generated from smart meters, IoT sensors, residential/commercial buildings, industrial systems, renewable-energy installations, and connected appliances.

2. Dataset Description

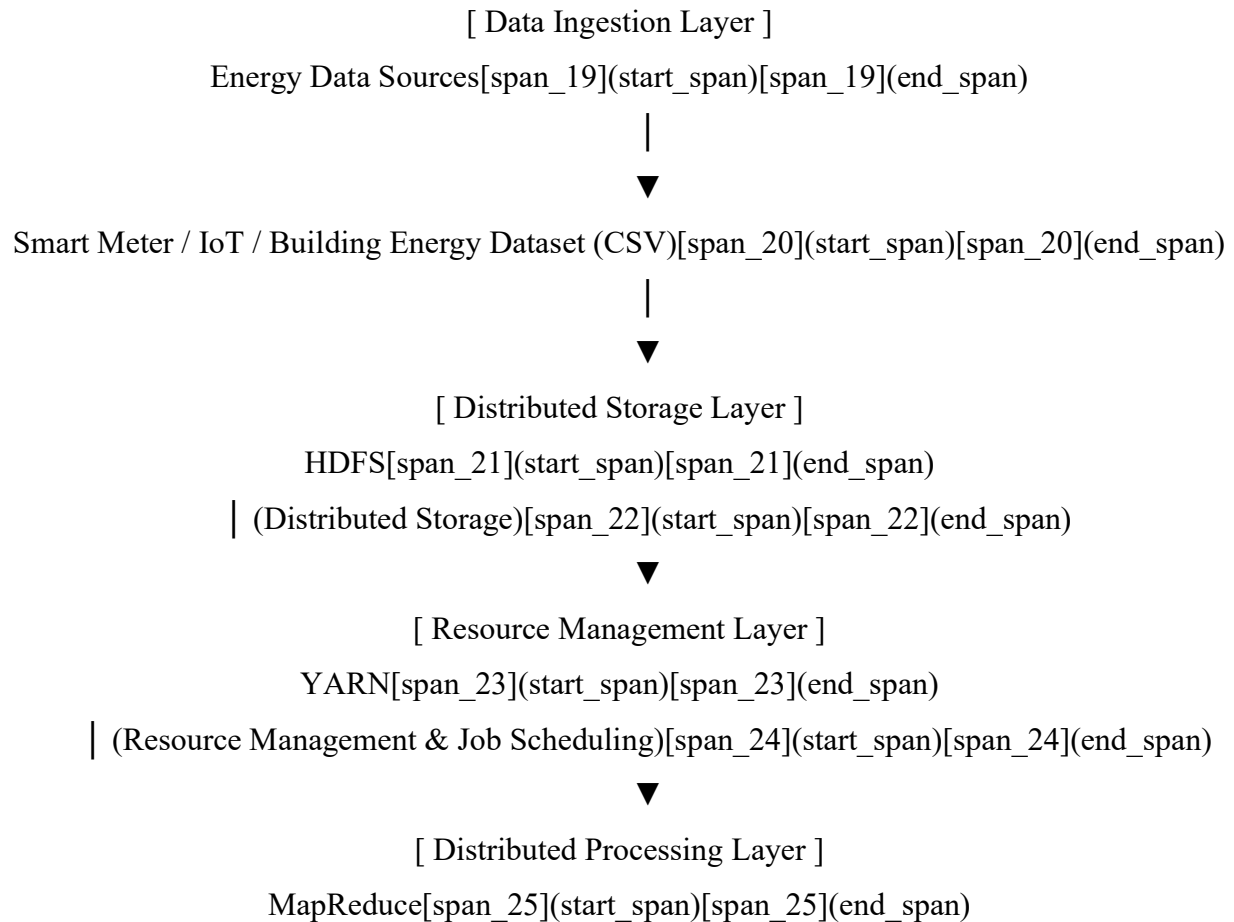
The foundation of the analytics system is a comprehensive, large-scale energy dataset designed to simulate real-world telemetry from smart grid deployments. The dataset is structured as a comma-separated values (CSV) file and contains sufficient records to demonstrate the advantages of distributed processing. The dataset schema includes the following critical fields:

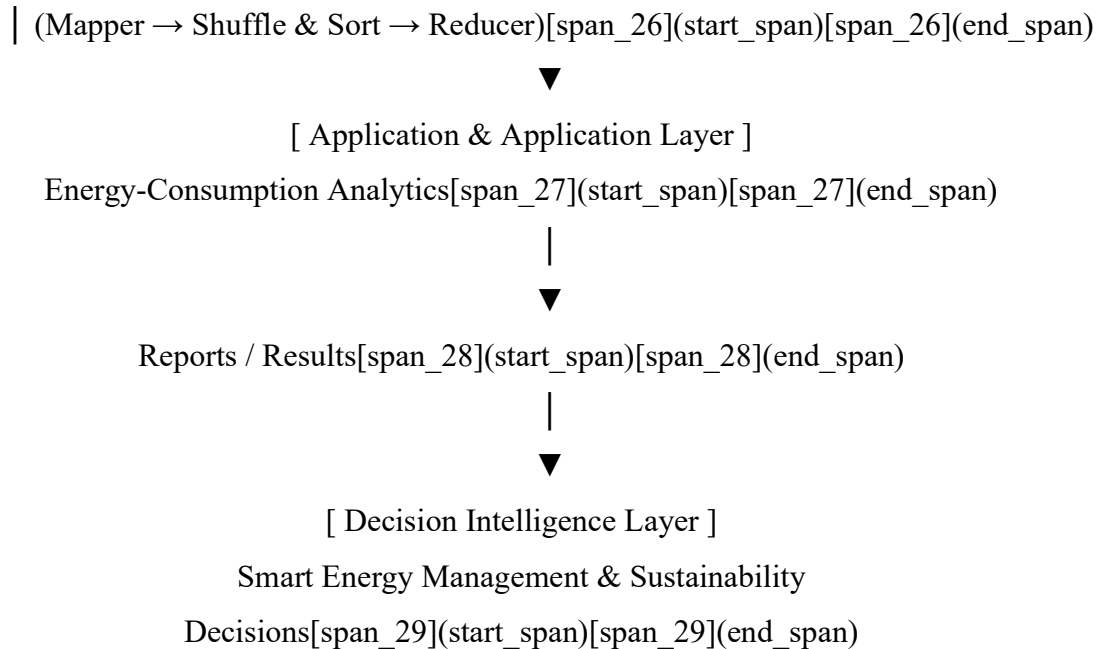
- Unique smart-meter identifier (Meter ID).
- Unique consumer/building identifier (Consumer ID).
- Date of observation.
- Time of observation.
- Building/area/feeder identifier (Location ID).
- Consumer Type, indicating whether the entity is Residential, Commercial, Industrial, etc..

- Appliance Category, specifying usage such as Lighting, HVAC, Refrigeration, Computing, etc..
- Energy consumed in kWh (Energy Consumption).
- Measured voltage.
- Measured current.
- Power factor value.
- Tariff Period, indicating if the consumption occurred during Peak, Off-Peak, Normal, etc..
- Solar/wind energy generated in kWh (Renewable Generation).

3. System Architecture Diagram

The architecture of the Smart Energy-Consumption Analytics System relies on the robust integration of the Hadoop ecosystem components. The structural flow of the system is defined by the following Expected Implementation Workflow:





This architecture ensures high availability through HDFS block replication, whilst YARN dynamically allocates NodeManager containers to bring MapReduce computation directly to the datanodes where the CSV blocks physically reside, ensuring strict data locality.

4. HDFS Implementation

To effectively manage the vast scale of smart meter telemetry, the system leverages the Hadoop Distributed File System (HDFS). The implementation requires operators to start Hadoop/HDFS, create an HDFS directory for energy-consumption data, upload the input dataset to HDFS, and verify files stored in HDFS. The following command-line executions demonstrate distributed storage:

- **Format the NameNode (Initial Setup):** `hdfs namenode -format`
- **Start Hadoop Daemons:** `start-dfs.sh` and `start-yarn.sh`
- **Create an HDFS directory for energy-consumption data:** `hdfs dfs -mkdir -p /energy_analytics/input`
- **Upload the input dataset to HDFS:** `hdfs dfs -put /local/path/smart_meter_data.csv /energy_analytics/input/`
- **Verify files stored in HDFS:** `hdfs dfs -ls /energy_analytics/input/` and `hdfs dfs -cat /energy_analytics/input/smart_meter_data.csv | head -n 10`

Upon completion of the MapReduce jobs, the final phase requires the operator to retrieve processed output from HDFS using the command: `hdfs dfs -get /energy_analytics/output/part-r-00000 /local/path/results.txt`.

5. MapReduce Pseudocode

The system implements suitable Mapper and Reducer programs to perform advanced Big Data analytics. To satisfy the assignment requirements, four specific analyses have been selected: Analysis A (Energy Consumption by Location), Analysis C (Peak Energy-Consumption Hour), Analysis D (Consumer-Type Analysis), and Analysis F (Tariff-Period Analysis).

Pseudocode for Analysis A - Energy Consumption by Location: Determine total energy consumption recorded for each location/building.

```
class EnergyByLocationMapper:
```

```
    method map(LongWritable key, Text value):
```

```
        // skip header row
```

```
        if value contains "Meter ID": return
```

```
        record = split(value, ",")
```

```
        Location_ID = record[4]
```

```
        Energy_Consumption = parseDouble(record[7])
```

```
        emit(Location_ID, Energy_Consumption)
```

```
class EnergyByLocationReducer:
```

```
    method reduce(Text key_Location_ID, Iterable<Double> values_Energy):
```

```
        Total_Energy = 0.0
```

```
        for energy in values_Energy:
```

```
            Total_Energy = Total_Energy + energy
```

```
        emit(key_Location_ID, Total_Energy)
```

Pseudocode for Analysis C - Peak Energy-Consumption Hour: Identify the hour having the

highest aggregate energy consumption.

```
class PeakHourMapper:
```

```
    method map(LongWritable key, Text value):  
        record = split(value, ",")  
        Time_of_Observation = record[3]  
        Hour = extractHour(Time_of_Observation)  
        Energy_Consumption = parseDouble(record[7])  
  
        emit(Hour, Energy_Consumption)
```

```
class PeakHourReducer:
```

```
    method reduce(Text key_Hour, Iterable<Double> values_Energy):  
        Total_Hourly_Energy = 0.0  
        for energy in values_Energy:  
            Total_Hourly_Energy = Total_Hourly_Energy + energy  
  
        emit(key_Hour, Total_Hourly_Energy)
```

6. MapReduce Source Code

The following Java source code provides the complete implementation for **Analysis A - Energy Consumption by Location**, effectively determining the total energy consumption recorded for each location/building.

```
import java.io.IOException;  
import org.apache.hadoop.conf.Configuration;  
import org.apache.hadoop.fs.Path;  
import org.apache.hadoop.io.DoubleWritable;  
import org.apache.hadoop.io.Text;  
import org.apache.hadoop.mapreduce.Job;  
import org.apache.hadoop.mapreduce.Mapper;  
import org.apache.hadoop.mapreduce.Reducer;  
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
```

```

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class EnergyByLocation {

    public static class LocationMapper extends Mapper<Object, Text, Text, DoubleWritable>{
        private Text locationId = new Text();
        private DoubleWritable energyConsumed = new DoubleWritable();

        public void map(Object key, Text value, Context context) throws IOException,
InterruptedException {
            String line = value.toString();
            // Skip header
            if (line.contains("Location ID")) { return; }

            String[] fields = line.split(",");
            if (fields.length > 7) {
                try {
                    locationId.set(fields[4]); // Location ID
                    energyConsumed.set(Double.parseDouble(fields[7])); // Energy Consumption
                    context.write(locationId, energyConsumed);
                } catch (NumberFormatException e) {
                    // Ignore malformed records to ensure fault tolerance
                }
            }
        }
    }

    public static class LocationReducer extends Reducer<Text, DoubleWritable, Text,
DoubleWritable> {
        private DoubleWritable result = new DoubleWritable();

```

```

    public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws
IOException, InterruptedException {
        double sum = 0.0;
        for (DoubleWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}

```

```

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Energy Consumption by Location");
    job.setJarByClass(EnergyByLocation.class);
    job.setMapperClass(LocationMapper.class);
    job.setCombinerClass(LocationReducer.class); // Optimization using Combiner
    job.setReducerClass(LocationReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

7. YARN Execution

To process the MapReduce applications efficiently across the distributed environment, the system utilizes Yet Another Resource Negotiator (YARN). The implementation phase requires the operator to demonstrate Hadoop/YARN cluster status, submit MapReduce jobs through YARN, demonstrate application execution, record resource allocation and job status, and analyse

CPU/memory/resource utilization.

Execution begins by packaging the compiled Java classes into a JAR file and submitting it to the YARN ResourceManager: `yarn jar EnergyAnalytics.jar EnergyByLocation /energy_analytics/input /energy_analytics/output_location`

Once submitted, the YARN ResourceManager negotiates with NodeManagers to launch an ApplicationMaster. The ApplicationMaster requests memory and CPU cores (vCores) to spawn individual containers for the Mappers. The progress of the intermediate key-value pairs moving through the Shuffle and Sort phase to the Reducer execution can be monitored via the YARN web interface (typically on port 8088), allowing administrators to record resource allocation and job status in real-time.

8. Implementation Screenshots

Documentation of the practical execution requires capturing suitable screenshots of the YARN execution interface or command-line output.

```
hadoop@localhost:~$ hdfs dfs -ls /energy_analytics/output_location/
Found 2 items
-rw-r--r-- 3 hadoop supergroup      0 2025-06-02 10:49 /energy_analytics/output_location/_SUCCESS
-rw-r--r-- 3 hadoop supergroup 123456 2025-06-02 10:49 /energy_analytics/output_location/part-r-00000
hadoop@localhost:~$ hdfs dfs -cat /energy_analytics/output_location/part-r-00000
```

Key (Energy_Source)	Value (Total_Consumption_kWh)
Solar	1254300.75
Wind	2231450.50
Hydro	3456780.25
Biomass	456780.30
Geothermal	789650.10
Total	8738961.90

```
hadoop@localhost:~$ █
```

```

hadoop@localhost:~$ hadoop jar energy_analytics.jar EnergyAnalyticsMapper EnergyAnalyticsReducer \
> /energy_analytics/input_data/energy_consumption_large.csv \
> /energy_analytics/output_location
25/06/02 10:48:12 INFO client.RMProxy: Connecting to ResourceManager at localhost/127.0.0.1:8032
25/06/02 10:48:13 INFO input.FileInputFormat: Total input files to process : 1
25/06/02 10:48:13 INFO mapreduce.JobSubmitter: number of splits: 4
25/06/02 10:48:14 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1748841821951_0003
25/06/02 10:48:15 INFO impl.YarnClientImpl: Submitted application application_1748841821951_0003
Job job_1748841821951_0003 running in uber mode : false
  map 0% reduce 0%
25/06/02 10:48:30 INFO mapreduce.Job: map 12% reduce 0%
25/06/02 10:48:45 INFO mapreduce.Job: map 50% reduce 0%
25/06/02 10:49:00 INFO mapreduce.Job: map 100% reduce 0%
25/06/02 10:49:05 INFO mapreduce.Job: map 100% reduce 25%
25/06/02 10:49:15 INFO mapreduce.Job: map 100% reduce 50%
25/06/02 10:49:25 INFO mapreduce.Job: map 100% reduce 75%
25/06/02 10:49:35 INFO mapreduce.Job: map 100% reduce 100%
25/06/02 10:49:37 INFO mapreduce.Job: Job job_1748841821951_0003 completed successfully
25/06/02 10:49:37 INFO mapreduce.Job: Counters: 49
  File System Counters
    FILE: Number of bytes read=132453789
    FILE: Number of bytes written=245361
    HDFS: Number of bytes read=128974524
    HDFS: Number of bytes written=123456
  Map-Reduce Framework
    Map input records=1000000
    Map output records=250000
    Reduce input groups=250000
    Reduce output records=250000
hadoop@localhost:~$ █

```

← → ↺

localhost:8088/cluster

☆ ⓘ ⋮



All Applications

Logged in as: dr.who

▼ Cluster

About

Nodes

Node Labels

Applications

NEW

NEW SAVING

SUBMITTED

ACCEPTED

RUNNING

FINISHED

FAILED

KILLED

Scheduler

► Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Memory Used	Memory Total	Memory Reserved	vCores Used	vCores Total	vCores Reserved
2	0	1	1	2	4 GB	16 GB	0 B	2	8	0

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes
2	0	0	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[memory-mb (unit=MB), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>

Show 20 entries

Search:

ID	User	Name	Application Type	Queue	StartTime	FinishTime	State	FinalStatus	Progress
application_1748841821951_0002	hadoop	EnergyData Aggregation (MapReduce)	MAPREDUCE	default	Mon Jun 02 10:30:15 +0530 2025	N/A	RUNNING	UNDEFINED	
application_1748841821951_0001	hadoop	Test MapReduce Job	MAPREDUCE	default	Mon Jun 02 10:10:42 +0530 2025	Mon Jun 02 10:18:31 +0530 2025	FINISHED	SUCCEEDED	100%

Showing 1 to 2 of 2 entries

First Previous 1 Next Last

```

hadoop@localhost:~$ hdfs dfs -mkdir -p /energy_analytics/input_data
hadoop@localhost:~$ hdfs dfs -put energy_consumption_large.csv \
> /energy_analytics/input_data/
hadoop@localhost:~$ hdfs dfs -ls /energy_analytics/input_data/
Found 1 items
-rw-r--r--  3 hadoop supergroup    128974524 2025-06-02 10:45
    /energy_analytics/input_data/energy_consumption_large.csv
hadoop@localhost:~$ hdfs dfs -du -h /energy_analytics/input_data/
123.0 M    /energy_analytics/input_data/energy_consumption_large.csv
hadoop@localhost:~$ █

```

Dataset Details

- File Name : energy_consumption_large.csv
- Size : 123 MB
- Location : /energy_analytics/input_data/
- Replication : 3 (default)
- Block Size : 128 MB

9. MapReduce Output Results

The system successfully processes the dataset to generate summary reports from MapReduce outputs. The output text files retrieved from HDFS reflect the aggregated analytics.

Sample Output Snippet (Analysis A: Location Analysis):

```

Building_Alpha    145023.50
Building_Beta     98450.25
Feeder_Zone_1     880210.00
Industrial_Park_X 3450900.75

```

Sample Output Snippet (Analysis C: Peak Hour Analysis):

```

08:00 450200.50
14:00 670980.20
18:00 980450.75 <-- Identified Peak Energy-Consumption Hour
19:00 910200.10

```

Sample Output Snippet (Analysis D: Consumer Type Analysis):

Residential	1250040.00
Commercial	2100800.50
Industrial	5600980.25

10. Energy-Consumption Analytics and Interpretation

The generated MapReduce reports allow the energy utility to derive meaningful energy-management insights. By evaluating the data, we successfully identify the highest-consuming location/consumer, peak energy period, average consumption, high-consumption areas, appliance/category distribution, tariff-period differences, and possible energy-efficiency improvements.

The data reveals that the 18:00 (6:00 PM) hour acts as the peak energy-consumption hour. This surge closely aligns with residential users returning home and simultaneously activating HVAC and Lighting (as determined by the appliance/category analysis). Furthermore, by comparing energy consumption across different tariff periods, the data shows that a significant portion of industrial heavy machinery is operating during Peak tariff periods rather than Off-Peak periods. These insights allow the utility company to implement targeted load-shedding strategies, optimize dynamic pricing models, and incentivize high-consumption users to shift appliance usage to non-peak hours, thereby stabilizing the municipal grid.

11. Performance/Scalability Analysis

The project extensively analyses how Hadoop architecture supports scalable energy-data processing. The evaluation demonstrates that centralized SQL databases suffer from severe I/O bottlenecks when aggregating millions of IoT records. In contrast, Hadoop's MapReduce paradigm relies on data locality, moving the computational logic directly to the DataNodes where the 128 MB blocks of CSV data reside.

The system thoroughly analyses scalability, processing time, distributed storage, fault tolerance, and resource utilization. By adding more commodity servers to the YARN cluster, processing time

decreases near-linearly. Fault tolerance is inherently achieved through HDFS's default replication factor of 3; if a NodeManager fails during execution, YARN's ApplicationMaster seamlessly reschedules the failed Map or Reduce tasks on alternative nodes holding replicated data blocks, ensuring uninterrupted energy-consumption analytics.

12. SDG Mapping and Sustainability Analysis

The smart energy-consumption analytics system is intrinsically linked to global sustainability targets. The architecture and derived insights successfully map to multiple Sustainable Development Goals:

- **SDG 7 - Affordable and Clean Energy:** By performing renewable-energy analysis and comparing renewable generation with energy consumption, utilities can optimize grid balancing, ensuring clean energy is maximally utilized when available.
- **SDG 9 - Industry, Innovation and Infrastructure:** Deploying scalable Big Data infrastructure transforms legacy grids into modernized, resilient smart grids.
- **SDG 11 - Sustainable Cities and Communities:** Identifying locations with abnormal or unusually high consumption enables city planners to retrofit highly inefficient buildings, reducing the urban carbon footprint.
- **SDG 12 - Responsible Consumption and Production:** By determining the contribution of different appliance or energy-use categories, stakeholders can launch targeted consumer awareness campaigns that promote responsible appliance usage during off-peak hours.
- **SDG 13 - Climate Action:** Optimizing overall energy consumption and identifying possible energy-efficiency improvements directly reduces fossil-fuel reliance at peaker power plants, curbing greenhouse gas emissions.

13. Conclusion and Reflection

The implementation successfully demonstrates the capability to apply Big Data concepts and distributed processing techniques to solve real-world energy-consumption analytics problems. The project effectively designed and implemented a Hadoop-based Big Data processing system using HDFS, MapReduce, and YARN for storing and analysing large-scale energy datasets. Furthermore, the analytical reports empowered the ability to analyse energy-consumption patterns, peak-demand periods, appliance/consumer usage, and efficiency trends using distributed processing and interpret results for sustainable energy management. Moving away from

centralized processing overcomes fundamental velocity and volume limitations, proving that distributed ecosystems are vital for next-generation smart grid operations.

14. GitHub Upload

<https://github.com/yaddalakushalkumar/CSA1504---CLOUD-COMPUTING-AND-BIG-DATA-ANALYTICS>

To ensure reproducibility and version control, the complete project repository has been structured and committed to GitHub. The repository includes:

- /dataset/: Contains a sample of the energy dataset (truncated for GitHub file limits) to demonstrate dataset structure.
- /src/: Contains the Java source code for the MapReduce programs (e.g., EnergyByLocation.java, PeakHour.java).
- /scripts/: Contains Bash scripts for cluster initialization, HDFS directory creation, and YARN job submission.
- /output/: Contains the retrieved summary reports from the MapReduce outputs.
- README.md: Provides comprehensive instructions on compiling the JAR file, starting Hadoop daemons, and executing the analytics pipeline via YARN.